AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI Conquer 2026

Technical Report (AI Conquer 2026)

Nhóm: Người cao tuổi

I. Tóm tắt báo cáo

- **Bài toán**: Hệ thống tự động phân loại cảm xúc đánh giá sản phẩm tiếng Việt thành ba nhãn Positive, Neutral và Negative nhằm thay thế việc đọc thủ công hàng nghìn đánh giá.
- **Dữ liệu**: dữ liệu đánh giá sản phẩm từ sàn thương mai điện tử như Shopee được thu thập, gán nhãn tự động theo điểm.
- **Hướng giải quyết**: Nhóm xây dựng pipeline theo kiến trúc ba lớp Streamlit UI, FastAPI backend, và mô hình học máy — triển khai trong Docker. Các mô hình được thử nghiệm: TF-IDF + Logistic Regression, PhoBERT, mBERT, tinyBERT.
- **Kết quả**: Mô hình Logistic Regression đạt Macro F1 = 0.637 và Accuracy đạt 72.74% vượt yêu cầu đề ra (F1 $\geq$ 0.6). PhoBERT đạt F1 cao nhất (0.685) nhưng yêu cầu phần cứng GPU để huấn luyện.
- **Kết luận**: Mô hình Logistic Regression có thể được chọn để triển khai production vì cân bằng giữa hiệu suất và chi phí hạ tầng. Hạn chế của mô hình là chưa phân loại chính xác các mẫu Neutral, chủ yếu do mẫu ít và ranh giới ngữ nghĩa mơ hồ, cần thu thập thêm dữ liệu và tối ưu mô hình trong các bước phát triển tiếp theo.

Mục lục

I. Tóm tắt báo cáo **1**

II. Giới thiệu **4**

III. Phát biểu bài toán **4**

IV. Dữ liệu **5**

V. Tiền xử lý dữ liệu **6**

V.1. Làm sạch bảng dữ liệu và chuẩn hóa nhãn 6

V.2. Chuẩn hóa văn bản 7

V.3. Đầu ra sau tiền xử lý 9

V.4. Encoding, tokenization và feature engineering 9

V.5. Chia tập dữ liệu 10

VI. Phương pháp / Cách tiếp cận **11**

VI.1. Ý tưởng tổng quát 11

VI.2. Phương pháp thực hiện 11

VII. Triển khai hệ thống **11**

VII.1. Cấu trúc hệ thống 11

VII.2. Các thành phần chính trong pipeline 12

VII.3. Công cụ và thư viện sử dụng 13

VIII. Kiểm thử **14**

VIII.1. Môi trường kiểm thử 14

VIII.2. Chiến lược kiểm thử 15

VIII.3. Kết quả kiểm thử 16

VIII.4. Kết luận tổng thể 17

IX. Thực nghiệm và đánh giá **17**

IX.1. Thiết lập thực nghiệm 17

IX.2. Metrics sử dụng 18

IX.3. Các mô hình được triển khai và kết quả 18

X. Phân tích lỗi và thảo luận **19**

XI. Kết luận và hướng phát triển **20**

XII. Tài liệu tham khảo **21**

Phụ lục **22**

A Phụ lục **22**

II. Giới thiệu

Sentiment Analysis (Phân tích cảm xúc) là quá trình sử dụng các kỹ thuật Xử lý ngôn ngữ tự nhiên (NLP), Học máy (ML) và phân tích dữ liệu để xác định và phân loại cảm xúc (tích cực, tiêu cực, trung tính) ẩn trong văn bản. Mục tiêu của Phân tích cảm xúc là hiểu cảm xúc và thái độ của người dùng hoặc cộng đồng đối với một sản phẩm, dịch vụ hoặc chủ đề nhất định. Trong khuôn khổ của warm-up, mục tiêu chính của dự án là để các thành viên làm quen với việc làm việc nhóm. Bên cạnh đó, nhóm cũng hướng đến đầu ra là API và giao diện UI để người dùng có thể nhập văn bản và đánh giá, phân loại cảm xúc dựa trên dữ liệu đầu vào. Về mặt ứng dụng, Sentiment Analysis có thể được dùng để:

- Theo dõi cảm xúc của khách hàng trên mạng xã hội để đánh giá mức độ hài lòng với thương hiệu hoặc sản phẩm.
- Phân tích phản hồi của khách hàng qua email, trò chuyện và đánh giá trực tuyến để cải thiện dịch vụ.
- Đo lường hiệu quả của các chiến dịch tiếp thị, dựa trên cảm tính của người dùng đối với quảng cáo và nội dung truyền thông.
- Đánh giá sản phẩm, dịch vụ mới, giúp doanh nghiệp điều chỉnh chiến lược phát triển dựa trên phản hồi của người dùng.
- Phân tích đối thủ cạnh tranh, để hiểu cảm nhận của khách hàng về sản phẩm, dịch vụ của đối thủ cạnh tranh.
- Theo dõi xu hướng thị trường và phát hiện các cơ hội kinh doanh dựa trên phản hồi tích cực hoặc tiêu cực về các sản phẩm tương tự.
- Cải thiện trải nghiệm của khách hàng bằng cách phát hiện các vấn đề cụ thể và phản hồi kịp thời.

III. Phát biểu bài toán

Sentiment Analysis (Phân tích cảm xúc):

- Đầu vào: văn bản tiếng Việt
- Đầu ra: phân loại cảm xúc: Tích cực, Tiêu cực, Trung tính
- Loại bài toán: classification
- Tiêu chí đánh giá chính:
  - Chỉ số đánh giá kỹ thuật: F1-Score
  - Độ tin cậy (%)
  - Hiệu suất: Tốc độ xử lý văn bản và khả năng xử lý khi khối lượng dữ liệu tăng lên.
  - Khả năng xử lý sắc thái ngôn ngữ: Phủ định, mỉa mai, ngữ cảnh và trung lập
  - Cấp độ phân tích: Document level, Sentence Level

IV. Dữ liệu

Dữ liệu được sử dụng trong dự án được tham khảo từ Kaggle. Đây là tập dữ liệu đánh giá sản phẩm bằng tiếng Việt thu thập từ sàn thương mại điện tử Shopee.

Tập dữ liệu gốc được tải lên Google Drive để lưu trữ, sau đó tiến hành tạo notebook trên `Collab` để tiến hành xử lý dữ liệu. Từ notebook, tập dữ liệu gốc được tải xuống từ Google Drive bằng thư viện gdown và lưu tạm dưới tên `data.csv`.

Tập dữ liệu ban đầu có 31,460 dòng và 4 cột. Các trường thông tin chính được mô tả trong Bảng 1.

Bảng 1: Các trường thông tin trong dữ liệu ban đầu

Trường dữ liệu	Ý nghĩa
`comment`	Nội dung nhận xét, đánh giá sản phẩm bằng tiếng Việt. Đây là trường văn bản chính được đưa vào mô hình.
`label`	Nhãn cảm xúc ban đầu, được gắn nhãn dựa nội dung nhận xét và điểm đánh giá sản phẩm
`rate`	Điểm đánh giá sản phẩm, hỗ trợ đối chiếu cảm xúc với mức độ hài lòng của người dùng.
`Unnamed: 3`	Cột thừa sinh ra trong quá trình lưu trữ dữ liệu, không mang thông tin hữu ích cho huấn luyện.

Kết quả khảo sát ban đầu cho thấy dữ liệu có ba lớp cảm xúc. Phân bố nhãn trước tiền xử lý được trình bày trong Bảng 8.

Bảng 2: Phân bố nhãn ban đầu

Nhãn ban đầu	Ý nghĩa	Số lượng mẫu
`POS`	Tích cực	20,093
`NEG`	Tiêu cực	6,669
`NEU`	Trung tính	4,698

Một số vấn đề được ghi nhận trong dữ liệu gốc gồm:

- Tồn tại cột thừa `Unnamed: 3`
- Có khả năng xuất hiện giá trị thiếu ở trường `comment` hoặc `label`
- Có các bình luận bị trùng

- Nhãn cảm xúc chưa được chuẩn hóa
- Nội dung văn bản còn nhiễu

Các loại nhiễu thường gặp bao gồm emoji, URL, thẻ HTML, dấu câu, ký tự đặc biệt, teencode, viết tắt tiếng Việt và hiện tượng lặp ký tự như "đẹpppp". Đây là các yếu tố có thể làm tăng kích thước từ vựng, gây nhiễu đặc trưng và làm giảm khả năng tổng quát hóa của mô hình.

Ngoài ra, phân bố lớp cho thấy dữ liệu bị lệch về lớp tích cực. Đây là đặc trưng thường gặp trong dữ liệu đánh giá thương mại điện tử, vì nhiều người dùng để lại đánh giá khi hài lòng với sản phẩm. Do đó, các bước tiền xử lý và chia dữ liệu cần giữ lại tỷ lệ các lớp để tránh làm sai lệch quá trình huấn luyện và đánh giá.

V. Tiền xử lý dữ liệu

Quy trình tiền xử lý được thiết kế nhằm biến dữ liệu đánh giá thô thành đầu vào sạch, nhất quán và phù hợp cho cả mô hình truyền thống lẫn mô hình BERT. Các bước chính gồm làm sạch bảng dữ liệu, chuẩn hóa nhãn, chuẩn hóa văn bản, tạo đặc trưng và chia tập huấn luyện/kiểm thử.

V.1. Làm sạch bảng dữ liệu và chuẩn hóa nhãn

Đầu tiên, các cột không cần thiết có tên bắt đầu bằng `Unnamed` được loại bỏ, sau đó hệ thống chỉ giữ lại ba trường có ý nghĩa là `comment`, `label` và `rate`. Các dòng thiếu `comment` hoặc `label` bị xóa vì không thể dùng để học quan hệ giữa văn bản đầu vào và nhãn cảm xúc. Những bình luận trùng lặp cũng được loại bỏ để hạn chế việc một nội dung giống nhau xuất hiện nhiều lần và làm lệch mô hình.

Làm sạch bảng dữ liệu

```
df = df.loc[:, ~df.columns.str.contains("^Unnamed")]

df = df[["comment", "label", "rate"]]

df = df.dropna(subset=["comment", "label"])
df = df.drop_duplicates(subset=["comment"])
```

Nhãn cảm xúc ban đầu được chuẩn hóa bằng cách chuyển về chữ hoa, loại bỏ khoảng trắng dư và chuyển đổi sang dạng đầy đủ: `POS` → `POSITIVE`, `NEG` → `NEGATIVE`, `NEU` → `NEUTRAL`

Các dòng có nhãn không thuộc ba nhóm trên bị loại bỏ để đảm bảo bài toán nhất quán là phân loại ba lớp.

Chuẩn hóa nhãn cảm xúc

```
df["label"] = df["label"].astype(str).str.upper().str.strip()

label_mapping = {
    "POS": "POSITIVE",
    "NEG": "NEGATIVE",
    "NEU": "NEUTRAL",
}

df["sentiment"] = df["label"].map(label_mapping)

df = df.dropna(subset=["sentiment"])
```

V.2. Chuẩn hóa văn bản

Từ trường `comment`, project tạo thêm trường `clean_comment`. Đây là phiên bản văn bản đã được chuẩn hóa và là đầu vào chính của các mô hình. Pipeline xử lý văn bản bao gồm:

- Chuyển toàn bộ văn bản về chữ thường để giảm trùng lặp từ vựng do khác biệt chữ hoa/chữ thường.
- Loại bỏ emoji, URL và thẻ HTML vì các thành phần này thường không ổn định trong dữ liệu review.
- Rút gọn ký tự lặp nhiều lần, ví dụ các chuỗi có một ký tự lặp từ ba lần trở lên được đưa về tối đa hai ký tự.
- Loại bỏ dấu câu, ký tự đặc biệt và chỉ giữ lại chữ cái tiếng Việt, chữ cái Latin, chữ số và khoảng trắng.
- Chuẩn hóa một số từ viết tắt và teencode tiếng Việt, ví dụ `ko`, `k`, `khong` thành "không"; `dc`, `đc` thành "được"; `sp` thành "sản phẩm".
- Xóa các token một ký tự và chuẩn hóa khoảng trắng dư.

Chuẩn hóa và làm sạch nội dung bình luận

```python
import re

VIET_ABBREVIATIONS = {
    "ko": "không", "k": "không", "kh": "không", "khong": "không",
    "dc": "được", "đc": "được", "dk": "được", "mk": "mình",
    "mh": "mình", "cx": "cũng",  "vs": "với", "r": "rồi", "rr": "rồi",
    "nha": "nhé", "nhe": "nhé", "ok": "ổn", "oke": "ổn", "okey": "ổn",
    "okie": "ổn", "sp": "sản phẩm", "sl": "số lượng", "gh": "giao hàng",
    "đh": "đặt hàng", "ad": "shop", "mn": "mọi người", "mng": "mọi người",
    "iu": "yêu", "thik": "thích", "thix": "thích", "wa": "quá",
    "wá": "quá", "qá": "quá", "vz": "vậy", "z": "vậy", "tks": "cảm ơn",
    "thks": "cảm ơn", "thanks": "cảm ơn", "ship": "giao hàng",
    "shop": "cửa hàng", "date": "hạn sử dụng", "2day": "hôm nay",
    "a": "anh", "e": "em", "sz": "size", "ng": "người",
    "bth": "bình thường", "nchung": "nói chung"
}

def normalize_abbreviations(text: str) -> str:
    words = text.split()
    return " ".join(VIET_ABBREVIATIONS.get(w.lower(), w) for w in words)

def normalize_repeated_chars(text: str) -> str:
    return re.sub(r"(.)\1{2,}", r"\1\1", text)

def remove_emoji(text: str) -> str:
    emoji_pattern = re.compile(
        "["
        "\U0001F600-\U0001F64F\U0001F300-\U0001F5FF"
        "\U0001F680-\U0001F6FF\U0001F1E0-\U0001F1FF"
        "\U00002700-\U000027BF\U0001F900-\U0001F9FF"
        "]+",
        flags=re.UNICODE,
    )
    return emoji_pattern.sub(" ", text)

def clean_text(text: str) -> str:
    if not isinstance(text, str) or text.strip() == "":
        return ""

    text = text.lower()
    text = remove_emoji(text)
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"<[^>]+>", " ", text)
    text = normalize_repeated_chars(text)
    text = re.sub(r"[^a-zA-Z0-9À-ỹ\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    text = normalize_abbreviations(text)
    text = re.sub(r"\b[a-zA-Z0-9]\b", " ", text)
    return re.sub(r"\s+", " ", text).strip()

df["clean_comment"] = df["comment"].apply(clean_text)
```

Sau bước này, các dòng có `clean_comment` rỗng tiếp tục bị loại bỏ vì không còn thông tin văn bản để đưa vào mô hình.

Xóa các dòng rỗng sau khi xử lý

```
df = df[df["clean_comment"].str.strip() != ""]
```

V.3. Đầu ra sau tiền xử lý

Sau khi làm sạch, tập dữ liệu còn 26,741 mẫu. Dữ liệu cuối cùng dùng cho huấn luyện gồm ba cột: `clean_comment`, `sentiment` và `rate`.

Bảng 3: Phân bố nhãn sau tiền xử lý

Nhãn sau chuẩn hóa	Số lượng mẫu
POSITIVE	16,183
NEGATIVE	6,321
NEUTRAL	4,237

Minh họa một số mẫu dữ liệu sau tiền xử lý.

Bảng 4: Ví dụ dữ liệu sau tiền xử lý

clean_comment	sentiment	rate
áo bao đẹp ạ	POSITIVE	5
hôm nay ao không giong trong	NEGATIVE	1
mùi thơm bôi lên da mềm da	POSITIVE	5

V.4. Encoding, tokenization và feature engineering

Với hướng tiếp cận TF-IDF, trường `clean_comment` được biến đổi thành đặc trưng văn bản bằng hai nhóm đặc trưng: word n-gram với khoảng n-gram từ 1 đến 3 và char n-gram với khoảng n-gram từ 3 đến 5. Việc kết hợp hai loại đặc trưng này giúp mô hình học được cả cụm từ có ý nghĩa và các mẫu ký tự nhỏ, hữu ích với tiếng Việt không dấu, teencode hoặc lỗi gõ nhẹ.

Với hướng tiếp cận BERT, nhãn cảm xúc được mã hóa thành số nguyên theo ánh xạ `NEGATIVE = 0`, `NEUTRAL = 1`, `POSITIVE = 2`. Văn bản được đưa qua tokenizer với độ dài tối đa `max_len = 128`, sử dụng padding để đưa các câu ngắn về cùng kích thước và truncation để cắt các câu dài vượt quá giới hạn. Đầu ra của bước tokenization gồm `input_ids`, `attention_mask` và `labels`.

Tạo đặc trưng và tokenization

```
from sklearn.feature_extraction.text import TfidfVectorizer

# TF-IDF features for traditional ML models
tfidf_word = TfidfVectorizer(
    analyzer="word", ngram_range=(1, 3),
    max_features=50000, sublinear_tf=True,
    min_df=2, max_df=0.95,
)
tfidf_char = TfidfVectorizer(
    analyzer="char_wb", ngram_range=(3, 5),
    max_features=30000, sublinear_tf=True,
    min_df=3,
)

# BERT label encoding and tokenizer input
LABEL_MAP = {"NEGATIVE": 0, "NEUTRAL": 1, "POSITIVE": 2}
df["label"] = df["sentiment"].map(LABEL_MAP)

enc = tokenizer(
    text,
    max_length=128,
    padding="max_length",
    truncation=True,
    return_tensors="pt",
)
```

V.5. Chia tập dữ liệu

Dữ liệu sạch được chia bằng hàm `train_test_split` với `test_size = 0.2`, `random_state = 42` và tham số `stratify`. Cách chia này tương ứng với 80% dữ liệu cho huấn luyện và 20% dữ liệu cho validation/test. Việc dùng `stratify` giúp duy trì tỷ lệ ba lớp cảm xúc trong cả hai tập, đặc biệt quan trọng vì dữ liệu ban đầu có xu hướng lệch về lớp tích cực.

Chia tập train và test

```
from sklearn.model_selection import train_test_split

X = df["clean_comment"]
y = df["sentiment"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

VI. Phương pháp / Cách tiếp cận

VI.1. Ý tưởng tổng quát

Dự án xây dựng một hệ thống Phân tích cảm xúc (Sentiment Analysis) cho các đánh giá sản phẩm bằng tiếng Việt, thu thập từ các nền tảng thương mại điện tử như Shopee. Hệ thống nhận đầu vào là một đoạn văn bản đánh giá của người dùng, sau đó phân loại cảm xúc thành một trong ba nhãn:

- Tích cực (Positive): Người dùng hài lòng với sản phẩm
- Trung tính (Neutral): Người dùng không có cảm xúc rõ ràng
- Tiêu cực (Negative): Người dùng không hài lòng hoặc phàn nàn

Sản phẩm cuối cùng bao gồm ba thành phần chính: pipeline huấn luyện mô hình, API phục vụ dự đoán xây dựng bằng FastAPI, và giao diện người dùng trực quan xây dựng bằng Streamlit. Toàn bộ hệ thống được đóng gói trong Docker để dễ dàng triển khai.

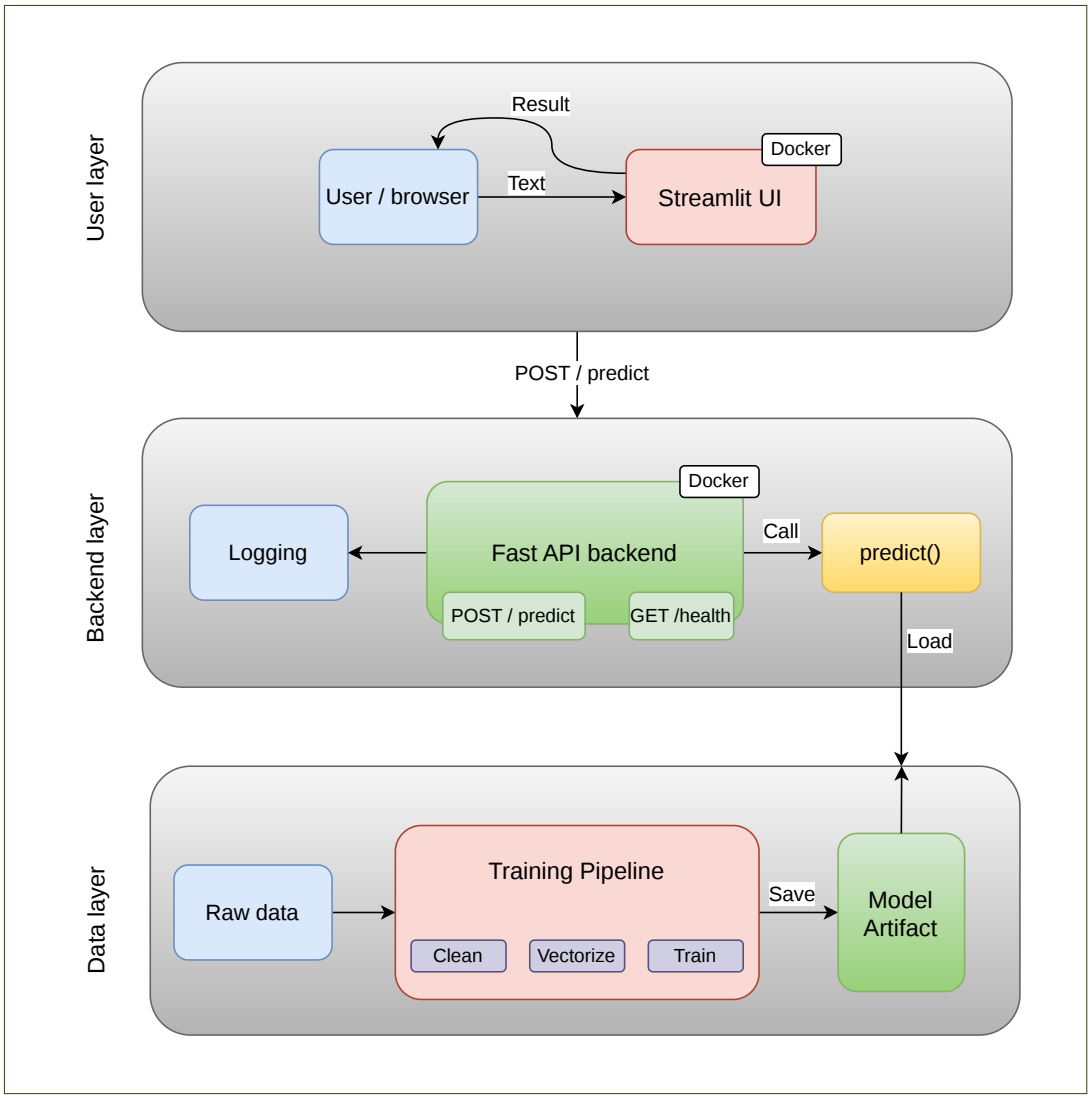
VI.2. Phương pháp thực hiện

- Giai đoạn 1 - Baseline: TF-IDF + Logistic Regression: Xây dựng mô hình đầu tiên làm tham chiếu để đánh giá các cải tiến về sau. Nhiệm vụ ở giai đoạn này là tập trung xử lý dữ liệu và tiến hành training thử.
- Giai đoạn 2 - Cải thiện mô hình, dữ liệu: Sau giai đoạn chạy thử, phân tích các lỗi về dữ liệu và mô hình, tiến hành triển khai test cases.
- Giai đoạn 3 - Mô hình nâng cao: Triển khai các mô hình nâng cao như PhoBERT và mBERT

VII. Triển khai hệ thống

VII.1. Cấu trúc hệ thống

Hệ thống được tổ chức theo kiến trúc ba lớp tách biệt, giao tiếp với nhau qua HTTP/REST API. Mỗi lớp có thể cập nhật hoặc thay thế độc lập mà không hoặc ít ảnh hưởng đến các lớp còn lại.



Hình 1: Cấu trúc hệ thống

VII.2. Các thành phần chính trong pipeline

Hệ thống có hai pipeline riêng biệt: pipeline huấn luyện (chạy offline một lần) và pipeline suy luận (chạy realtime mỗi request).

VII.2.1. Pipeline huấn luyện

- Bước 1: Thu thập dữ liệu
  - Shopee reviews -> data raw
  - Cột comment, label, rate
- Bước 2: Gán nhãn
  - rate 4-5 -> POSITIVE
  - rate 3 -> NEUTRAL

  - rate 1-2 -> NEGATIVE
  - -> labeled data
- Bước 3: Làm sạch văn bản
  - xóa URL, ký tự đặc biệt, emoji
  - chuẩn hóa khoảng trắng
  - xóa bản ghi trùng lặp
  - -> cleaned data
- Bước 4: Huấn luyện
- Bước 5: Đánh giá Accurary, Macro F1
- Bước 6: Lưu model artifact

VII.2.2. Pipeline suy luận

Input: văn bản đánh giá (chuỗi ký tự)

- Bước 1: Validate input và tiền xử lý dữ liệu: độ dài, khoảng trắng,...
- Bước 2: Dự đoán
- Bước 3: Trả kết quả

Output: Streamlit hiển thị kết quả

VII.3. Công cụ và thư viện sử dụng

VII.3.1. Ngôn ngữ và môi trường

Bảng 5: Ngôn ngữ và môi trường

Công cụ	Phiên bản	Vai trò
`Python`	>= 3.10	Ngôn ngữ chính
`pip + venv`	_	Quản lý môi trường ảo
`Docker`	_	Đóng gói và triển khai
`Git`	_	Quản lý mã nguồn

VII.3.2. Thư viện sử dụng

Bảng 6: Thư viện sử dụng chính

Thư viện	Vai trò
`Pandas`	Đọc, làm sạch, lưu CSV
`scikit-learn`	TF-IDF, Logistic Regression, metrics
`joblib`	Lưu và load model/vectorizer
`torch`	Deep learning backend cho BERT
`fastapi`	Framework xây dựng REST API
`uvicorn`	ASGI server chạy FastAPI
`streamlit`	Web UI giao diện người dùng
`matplotlib`	Vẽ confusion matrix, biểu đồ phân phối

VIII. Kiểm thử

VIII.1. Môi trường kiểm thử

- Ngôn ngữ: Python 3.10 / 3.11
- PyTorch + Transformers
- Docker + GitHub Actions CI
- pytest (API testing)
- Playwright (UI testing)

Cấu hình CI Mode

- epochs = 1
- batch_size $\leq$ 8
- MAX_TRAIN = 50
- MAX_TEST = 20

Mục tiêu CI: chỉ kiểm tra pipeline stability, không phản ánh performance thật.

VIII.2. Chiến lược kiểm thử

Hệ thống được kiểm thử theo hướng phân tầng (layered testing), bao gồm mô hình, API, giao diện người dùng và toàn bộ pipeline CI/CD.

VIII.2.1. Kiểm thử mô hình (Model Testing)

Mục tiêu: đánh giá chất lượng dự đoán và độ ổn định của các mô hình TF-IDF và BERT.

- Đánh giá Accuracy và Macro-F1 trên tập test.
- Kiểm tra khả năng dự đoán đúng ngữ nghĩa (semantic correctness).
- Kiểm tra batch inference với nhiều input cùng lúc.
- Kiểm tra reproducibility (cùng input → cùng output).
- Kiểm tra edge cases: input rỗng, dài, nhiễu, tiếng Anh.

Kết quả thực tế:

- TF-IDF: PASS toàn bộ tiêu chí (Accuracy, F1, batch, edge cases). Mô hình ổn định và đạt ngưỡng đánh giá.
- BERT: FAIL Accuracy và Macro-F1 trên tất cả checkpoint; xuất hiện lỗi semantic và xu hướng lệch nhãn NEUTRAL.

VIII.2.2. Kiểm thử API

Mục tiêu: đảm bảo API hoạt động đúng logic và ổn định trong mọi tình huống.

- Kiểm thử các endpoint `/predict`, `/batch`, `/health`.
- Kiểm tra format request/response.
- Kiểm tra xử lý lỗi với input không hợp lệ.
- Kiểm tra consistency giữa API output và model output.

VIII.2.3. Kiểm thử giao diện người dùng (UI Testing)

Mục tiêu: đảm bảo UI hiển thị chính xác kết quả từ API.

- Kiểm tra hiển thị sentiment prediction.
- Kiểm tra luồng UI → API → Model.
- Kiểm thử E2E bằng Playwright.

Lưu ý: Kiểm thử UI hiện tại chưa hoàn thiện, chỉ có 1 test case cơ bản. Chưa đủ coverage cho các tình huống lỗi và tương tác người dùng.

VIII.2.4. Kiểm thử pipeline CI/CD

Mục tiêu: đảm bảo toàn bộ hệ thống hoạt động ổn định trong môi trường CI.

- Regression testing với dataset rút gọn.
- Kiểm tra Docker-based execution.
- Phát hiện lỗi sớm trong pipeline build/test.
- Đảm bảo không crash trong GitHub Actions CI.

VIII.3. Kết quả kiểm thử

VIII.3.1. Kết quả mô hình TF-IDF

- Accuracy ≥ 0.70: PASS
- Macro-F1 ≥ 0.60: PASS
- Batch inference: ổn định (10 samples OK)
- Edge cases: không crash
- Tốc độ suy luận: 1.2ms

Kết luận: TF-IDF là baseline ổn định và sẵn sàng production.

VIII.3.2. Kết quả mô hình BERT

- PhoBERT: Accuracy 0.45 / F1 0.31 (FAIL)
- mBERT: Accuracy 0.40 / F1 0.19 (FAIL)
- Demo: Accuracy 0.25 / F1 0.13 (FAIL)

Quan sát lỗi:

- Sai lệch dự đoán sang nhãn NEUTRAL
- Sai semantic trong nhiều trường hợp test
- Không crash $\rightarrow$ chỉ lỗi chất lượng

Kết luận: BERT chưa đạt chất lượng mô hình.

VIII.3.3. Kết quả kiểm thử API

- Total test cases: 175
- PASS: 113
- FAIL: 62
- Pass rate: 64.57%

Lỗi chính:

- /predict inference inconsistency
- mapping label sai logic
- validation input chưa chặt

VIII.3.4. Kết quả kiểm thử UI

- 1 test case (Playwright E2E)
- PASS

Lưu ý: UI mới ở mức cơ bản, chưa đủ test coverage.

VIII.4. Kết luận tổng thể

- TF-IDF: baseline ổn định, production-ready.
- BERT: chưa đạt chất lượng.
- API: mức trung bình (64.57%).
- UI: còn rất hạn chế.
- Hệ thống: phù hợp demo, chưa production.

IX. Thực nghiệm và đánh giá

IX.1. Thiết lập thực nghiệm

- Môi trường thực nghiệm
  - Ngôn ngữ: Python 3.10
  - Hệ điều hành: Windows 11, Ubuntu 24.04
  - Phần cứng BERT: Google Colab T4 GPU
  - Dữ liệu thực nghiệm: Dữ liệu đã trình bày ở mục IV.
- Quy trình thực nghiệm: Ba giai đoạn như đã trình bày ở VI.2

IX.2. Metrics sử dụng

Macro F1-score- Metrics quan trọng nhất. **Accuracy** - Tỉ lệ dự đoán đúng trên tổng số mẫu.
Ngưỡng đạt yêu cầu

Bảng 7: Metrics

Metric	Ngưỡng tối thiểu	Lý tưởng
Accuray	$\geq 70\%$	$\geq 75\%$
Macro F1	$\geq 60\%$	$\geq 65\%$

IX.3. Các mô hình được triển khai và kết quả

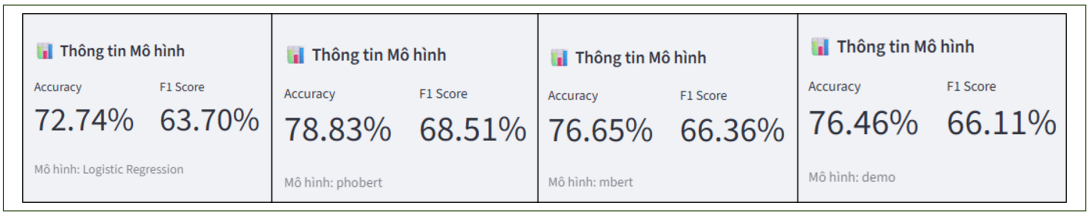
Có 4 mô hình được triển khai:

- Baseline: TF-IDF + Logistic Regression
- PhoBERT (tối ưu cho tiếng Việt)
- mBERT (multilanguage)
- Demo (Tiny BERT)

Kết quả tổng hợp trên data test:

Bảng 8: Kết quả model

Model	Accurray	Macro F1
Logistic Regression	72.74%	63.70%
PhoBERT	78.83%	68.51%
mBERT	76.65%	66.36%
TinyBERT	76.46%	66.11%



Hình 2: Tổng hợp kết quả model hiển thị trên UI

Nhận xét về kết quả:

- Baseline (Logistic Regression) đã ở mức chấp nhận được, đạt Macro F1 = 0.637 ngay từ lần đầu, cho thấy dữ liệu được làm sạch và gán nhãn tương đối tốt.
- PhoBERT có ưu thế hơn với mức Macro F1 = 0.685 cao nhất trong các model. Kết quả này khẳng định rằng với tiếng Việt, mô hình được huấn luyện trên dữ liệu đủ lớn và đúng ngôn ngữ có lợi thế rõ ràng.

X. Phân tích lỗi và thảo luận

Hệ thống đã thực hiện được:

- Các model có thể xử lý và phân loại cảm xúc cơ bản, trong đó PhoBERT được đánh giá tốt hơn với độ tin cậy cao nhất trong các model.
- Giao diện UI để người dùng nhập dữ liệu đánh giá.

Hạn chế và vấn đề còn tồn tại:

- Lớp Neutral vẫn là thách thức lớn nhất. Nguyên nhân: ranh giới giữa Neutral và Positive/Negative thường mang tính chủ quan và phụ thuộc ngữ cảnh mà TF-IDF không thể nắm bắt tốt.
- Các mô hình BERT tốn nhiều thời gian training và phải cấu hình các thông số train hợp lý mới cho ra kết quả đạt yêu cầu. Với môi trường CI, phần cứng hạn chế, các mô hình BERT gần như không thể triển khai.
- Tập dữ liệu còn hạn chế: Chưa đủ để mô hình tổng quát hóa tốt trên mọi loại đánh giá — đặc biệt là ngôn ngữ mỉa mai, từ lóng, hay đánh giá từ các lĩnh vực sản phẩm khác nhau.
- Mất cân bằng nhãn ảnh hưởng đến độ tin cậy: tỉ lệ mẫu Positive/Negative/Neutral không đồng đều làm ảnh hưởng đến kết quả dự đoán. Cần thu thập thêm dữ liệu Neutral và Negative để đánh giá chính xác hơn.

Phân tích lỗi tiêu biểu:

- **Nhóm 1: Phủ định kép (TF-IDF không xử lý được):**
  - Input: "Không phải không tốt, nhưng cũng không hẳn là hài lòng"
  - Dự đoán: Negative
  - Thực tế: Neutral
  - Nguyên nhân: TF-IDF mất thông tin về "không phải không"
- **Nhóm 2: Mỉa mai (cả TF-IDF và PhoBERT đều khó):**
  - Input: "Ôi chà, giao hàng nhanh thật, mất có 3 tuần thôi!"
  - Dự đoán: Neutral (TF-IDF), Positive (PhoBERT)
  - Thực tế: Negative

  - Nguyên nhân: Không nhận ra giọng mỉa mai

- **Nhóm 3: Đánh giá hỗn hợp:**
  - Input: "Hộp bị móp, shop cần cải thiện đóng gói, sản phẩm tốt"
  - Dự đoán: Neutral (TF-IDF), Negative (PhoBERT)
  - Thực tế: Neutral
  - Nguyên nhân: Từ 'cải thiện' và 'móp' lấn át 'tốt'

XI. Kết luận và hướng phát triển

Nhóm đã triển khai và so sánh các mô hình dùng cho phân loại cảm xúc ngôn ngữ tiếng Việt. Với kết quả so sánh, mô hình TF-IDF Logistic Regression có thể được chọn để sử dụng cho production vì đáp ứng các tiêu chí Macro F1 = 0.637 (ngưỡng chấp nhận được), kích thước mô hình nhỏ và không cần GPU. Bên cạnh đó, nhóm cũng đã xây dựng UI trực quan để người dùng nhập dữ liệu đánh giá và test hệ thống.

Hạn chế: Chưa phân loại được một số văn bản mang giọng điệu mỉa mai hoặc hỗn hợp. Dữ liệu chưa cân bằng khiến cho mô hình phân loại chưa chính xác.

Hướng cải thiện:

- Thu thập thêm mẫu dữ liệu, đặc biệt là mẫu Neutral
- Xử lý phủ định với rule-based
- Tinh chỉnh các tham số của mô hình TF-IDF + Logistic Regression
- Tăng cường dữ liệu, đảm bảo tổng quát hóa trên dữ liệu đánh giá
- Xem xét thêm khía cạnh tốc độ dự đoán cho các mô hình, đảm bảo cân bằng accuracy và speed.

XII. Tài liệu tham khảo

[1] FPT AI. "What is sentiment analysis in nlp? how it works and benefits", Accessed: May 1, 2026. [Online]. Available: https://fpt.ai/blogs/sentiment-analysis/.

[2] scikit-learn.org. "Sklearn.linear_model, logisticregression", Accessed: May 1, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

[3] Scikit-learn.org. "Sklearn.metrics, f1_score", Accessed: May 1, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html.

[4] Coursera. "What is the bert model and how does it work?", Accessed: May 1, 2026. [Online]. Available: https://www.coursera.org/articles/bert-model.

Phụ lục

A Phụ lục

- Link GitHub: https://github.com/AIVIETNAM-AIO-thanhnhan/SentimentAnalysis
- Video Demo: https://www.youtube.com/watch?v=9mZvn0qgZQU